UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

DEPARTMENT OF INFORMATION COMMUNICATION AND TECHNOLOGY

---------***---------

Machine Learning and Data Mining 2

Labwork 2 Report

T2 – Clustering

Members:

Trần Hiến Chương – 22BA13056

Lê Đức An – 22BA13002

Ha Noi, January 12, 2026

Contents

I. K-means clustering on datasets 2

1. Land Mine Dataset 2

1.1 Run experiments with k-means. Explain the experimental protocol..... 2

1.2. Explain the centroid initialization of the k-means used in the first question. 3

1.3. Analyze and compare the results with different values of k. 6

1.4. Calculate the clustering quality. 8

2. Gas Turbine CO and NOx Emission Data Set 10

2.1. Run experiments with k-means. Explain the experimental protocol..... 10

2.2. Explain the centroid initialization of the k-means used in the first question. 10

2.3. Analyze and compare the results with different values of k. 11

2.4. Calculate the clustering quality. 12

II. Subspace clustering 13

1. Dataset Overview 13

2. Use PCA or SVD to visualize the data distribution in 2D/3D. 13

3. Apply clustering method K-means for the selected datasets in 2D/3D after using PCA/SVD. Compare the performance before and after the dimensionality reduction. 14

4. Select a random subspace of the dataset. Re-apply any clustering method on the subspace.16

III.REFERENCES..... 19

LIST OF ABBREVIATIONS 19

I. K-means clustering on datasets

1. Land Mine Dataset

The Land Mines dataset [1] contains 2225 instances, each representing measurements from a simulated environment designed to mimic the challenges of landmine detection. Each instance comprises 9 features extracted from sensor readings. These features capture information about the simulated terrain and potential presence of landmines. As this is a simulated dataset, we will use it to examine the performance of K-means clustering.

1.1 Run experiments with k-means. Explain the experimental protocol.

For this dataset, we implemented the K-mean algorithm for Clustering tasks. We decided to run an experiment on a fixed value of K, which is picked by the Elbow Method and the DBI method.

Here is some information about the metrics that we use to evaluate clustering models:

- Inertia: Metric refers to the Within Cluster Sum of Square (WCSS). It measures the total distance between each datapoint and the initial centroids. WCSS with the smaller value means that the clusters are more compact and well separated.

- Davies-Bouldin Index (DBI): It assesses how well-separated the clusters are

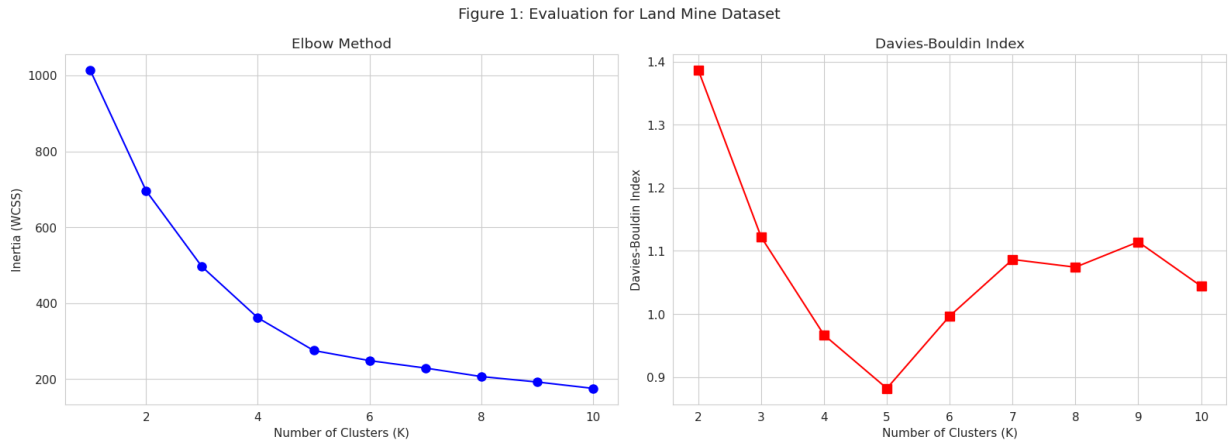

Figure 1: Total Within Sum of Squares of the landmine dataset

As shown in Figure 1, when observing the inertia value, the Elbow method was implemented. Somewhere at the value of K=5, the curve starts losing dramatic gains. Also with the DBI method, the lowest point is at K=5. Then, we ended up with K=5 for the number of clusters in this dataset.

1.2. Explain the centroid initialization of the k-means used in the first question.

In the K-Means algorithm, the selected Initial Centroids (ICs) can greatly affect the results. In this experiment, we make a comparison between 2 ways of choosing ICs which are Randomly generated and the K-Means++ algorithm.

K-Means++ Algorithm, developed by Arthur and Vassilvitskii (2007), aims to improve initialization by pushing the centroids as far as possible from one another, making convergence faster. The steps are:

1. Choose an IC_0 randomly from the data points.
2. Iteratively calculate the squared Euclidean distance $D(x_i)^2$ between all data points and the selected centroid.
3. Choose the next centroid c, by selecting a data point x_i withs probability proportional to $D(x_i)^2$. Points farther away are more likely to be selected.
4. Repeat until all K centroids are chosen.

Having done the PCA process on this dataset, we plotted the figure to visualize data distribution in 2D of the first two PCs, with the 5 ICs highlighted.

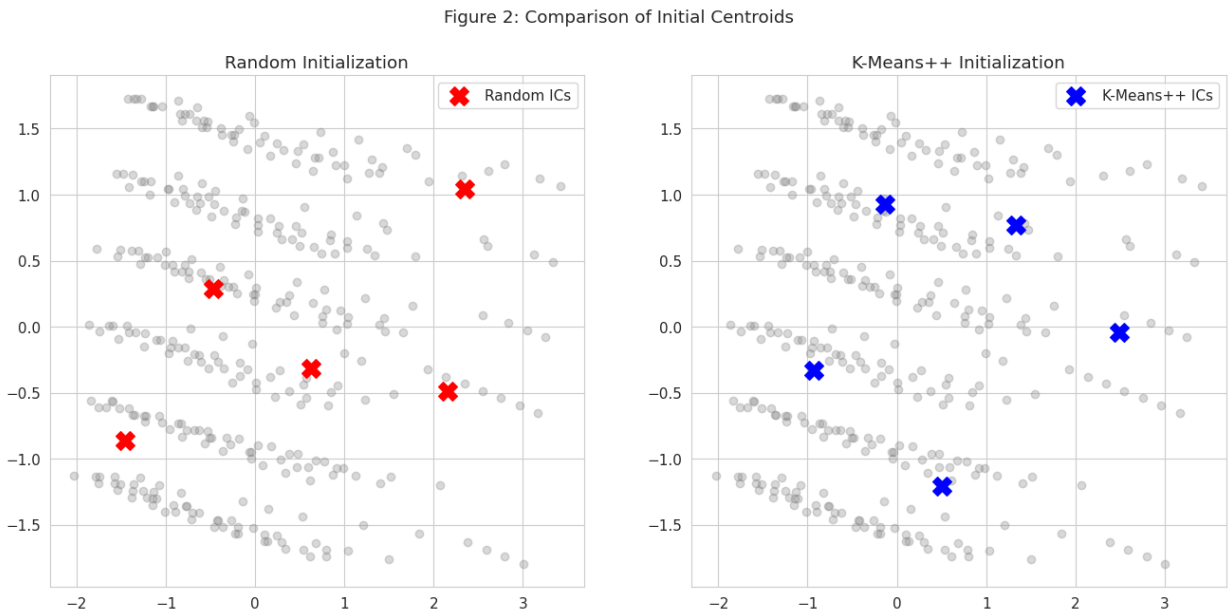

Figure 2: Comparison data distribution 2D with highlighted IC

As shown in Figure 2, with the random initialization method, ICs seem to be closer than with the K-means++ initialization method. This indicates that K-means++ generates better ICs. We calculated the WCSS of each particular IC as shown below:

Table 1: WCSS of ICs of two initialization methods

IC	WCSS with Random Initialization	WCSS with K-means++ Initialization
1	32.69	58.70
2	39.14	118.13
3	88.17	51.72
4	124.56	71.26
5	149.23	59.98
Total	433.78	359.79

From the table, we can confirm that the WCSS of the K-means++ method is smaller, which means that the ICs were selected more accurately. We were satisfied with this result, so we chose the K-means++ method to initialize ICs.

After selecting K with some metrics, we start the clustering process with the selected K, the K-Means++ algorithm and the data, which has been standardized. All the information is set into the Kmean function in Python, after some retry, we end up with 11 for the number of max iteration. Then

the figure can be plot as following:

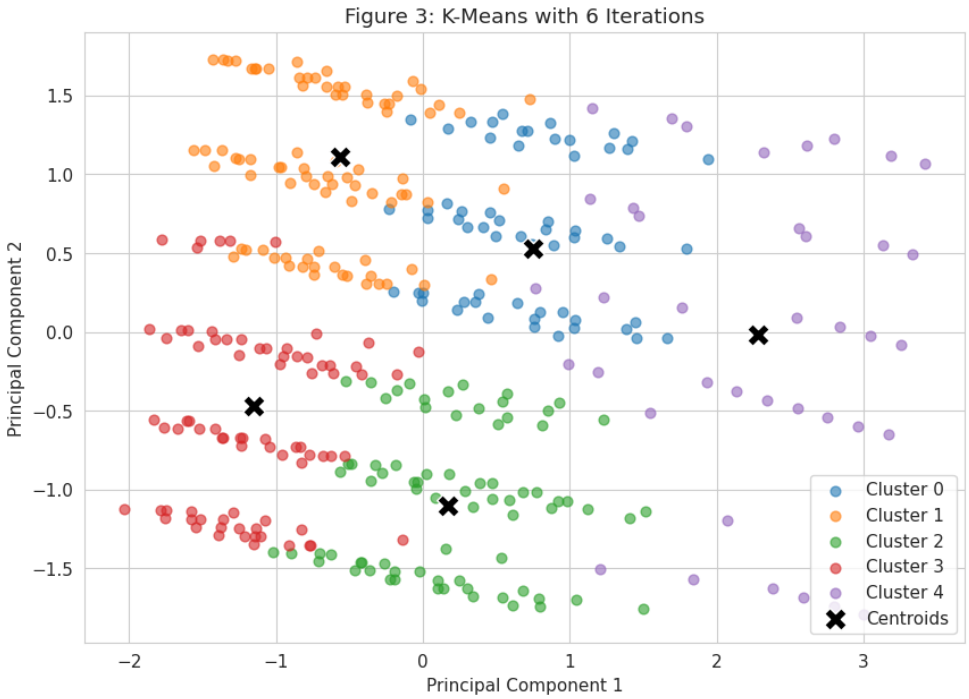

Figure 3: K-Means with 6 iterations

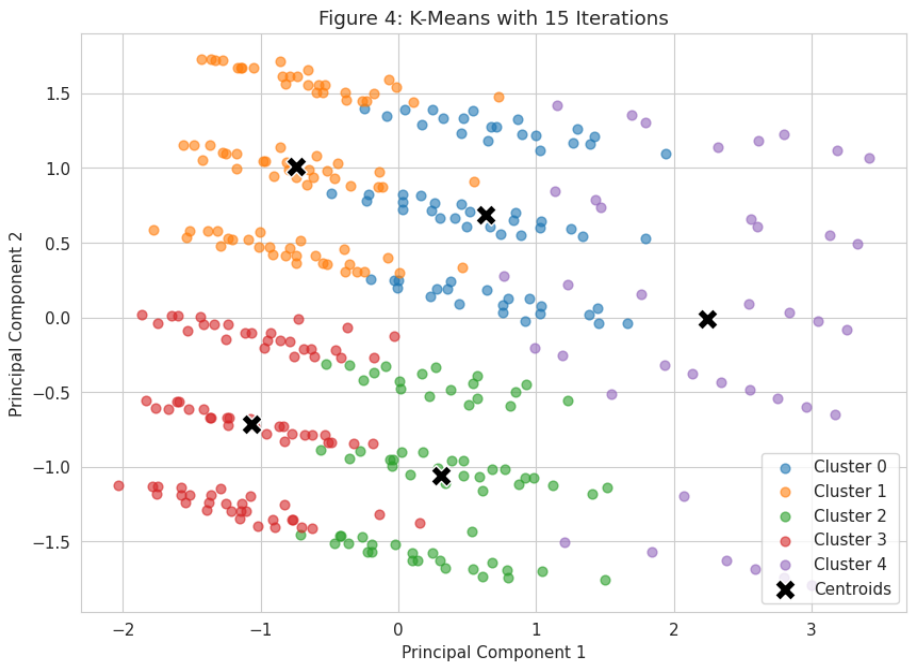

Figure 4: K-Means with 15 iterations

The two figures show the cluster progress by iterations. Initially, as the algorithm updates the cluster centroids and reassigns data points, the inertia which measures the sum of squared distances

between points and their nearest centroid-decreases significantly. However, by the 11th iteration, the Inertia stops decreasing. This indicates that the algorithm has reached a state of convergence and further iteration will not improve the cluster compactness.

By that information, we also plot the cluster result in 3D as following:

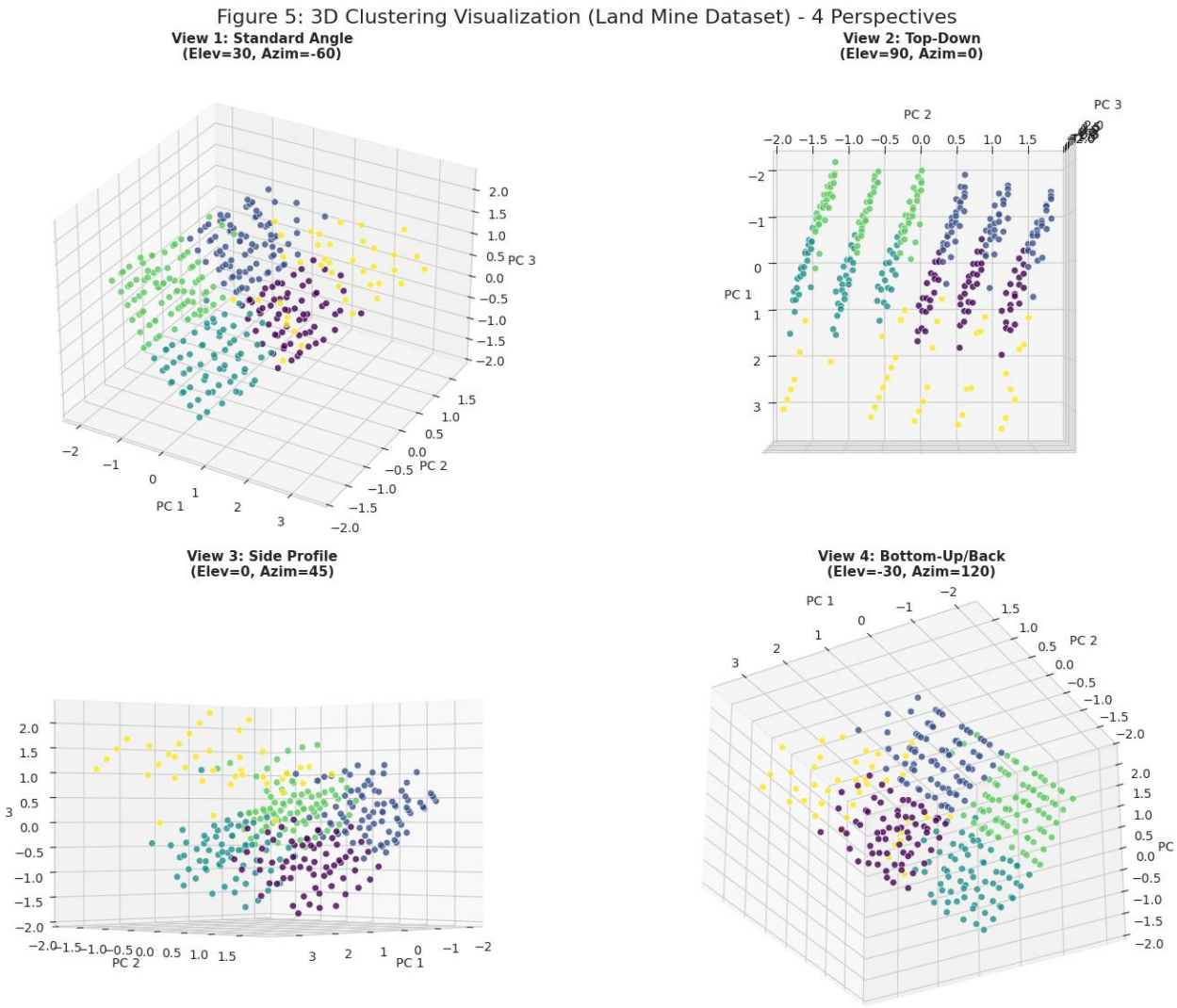

Figure 5: Clustering 3D Visualization

1.3. Analyze and compare the results with different values of k.

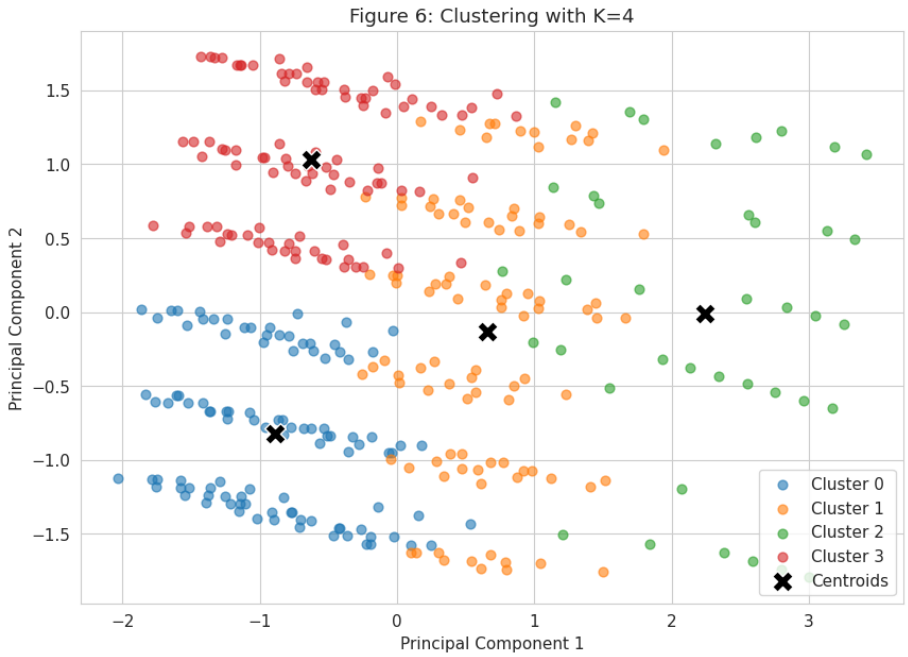

Figure 6: K-Means Clustering with K=4

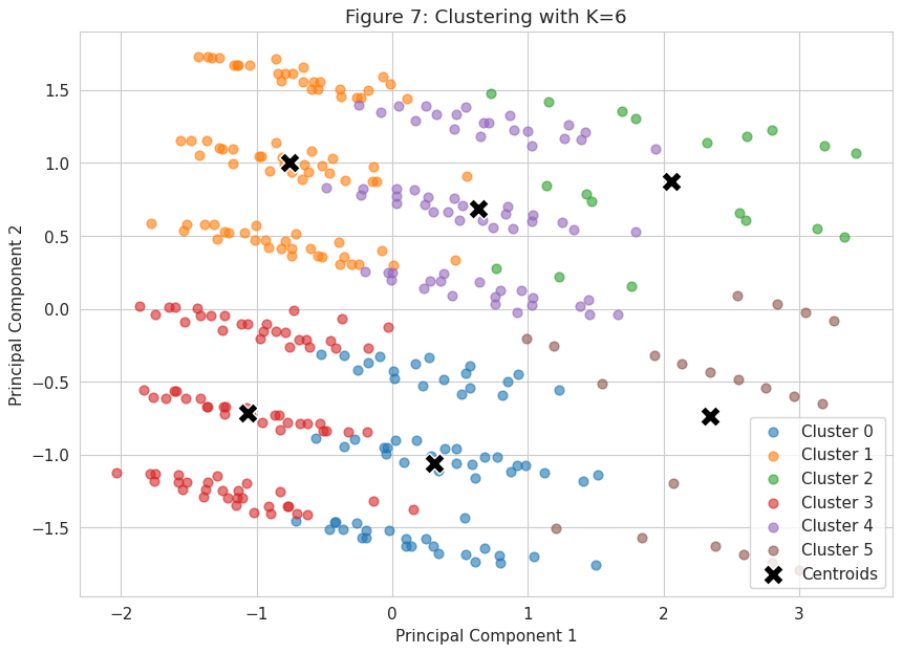

Figure 7: K-Means Clustering with K=6

From two different values of K which are 4 and 6, we can also see the data distribution of each clustering process result with different values of K. With K=4, the cluster seems to be imbalanced, where the compactness of the cluster with the centroid 1 is significantly smaller than all the other clusters. And also cluster with centroid 6 at the result of K=6. For more numerical evaluation, we calculate more detailed values as shown in the following table.

Table 2: Detail measurement values with various K

K Values	Cluster Sizes	Davies-Bouldin Index	Dunn Index
k=4	39, 104, 96, 99	1.3069	0.0039
k=5	82, 66, 84, 39, 67	1.2241	0.0043
k=6	81, 67, 65, 81, 26, 18	1.2940	0.0037

As shown in the table above, with k=5 achieve the lowest value of 1.2241, present that it has the best overall separation and compactness based on this metric. Moreover, analyzing the Dunn Index, it shows a similar trend where k=5 has the highest Dunn Index, indicating the best performance in terms of maximizing the ratio of minimum inter-cluster distance to maximum intra-cluster distance.

With those measurements and cluster size, k=6 shows good potential for overfitting with two small clusters; it doesn't perform better than k=5 when we consider DBI and Dunn Index. Even though k=4 has a more balanced cluster size, it's the worst performance according to DBI and Dunn Index. Overall, k=5 is the optimal choice.

1.4. Calculate the clustering quality.

To calculate the clustering quality, we implement the Davies-Bouldin Score (DB) which is an internal validation that measures the correlation between each cluster with its most similar cluster. The lower DB score, the better clustering because it reflects lower intra-cluster and higher inter-cluster separation.

The table below shows the k-means clustering results using k=5 under different max iteration settings (8, 11, 15). As shown, the algorithm only converged after 8 actual iterations in spite of max_iter being set to 8, 11 or 15.

Table 3: K-means clustering results (k=5) with vary max iter

max_iter	Inertia	Davies-Bouldin Score
8	275.53	0.88
11	275.33	0.88
15	275.33	0.88

These results show that the clustering quality is stable around an Inertia of 275.33 and DB score of 0.88, representing that the clusters are well-separated. Trying to increase the maximum number of iteration beyond 8 does not improve the cluster configuration.

2. Gas Turbine CO and NOx Emission Data Set

"The dataset contains 36733 instances of 11 sensor measures aggregated over one hour (by means of average or sum) from a gas turbine located in Turkey's north western region for the purpose of studying flue gas emissions, namely CO and NO_x ($NO+NO_2$)." [2] As this dataset has the first column "year", we need to drop it in the data preprocessing.

2.1. Run experiments with k-means. Explain the experimental protocol.

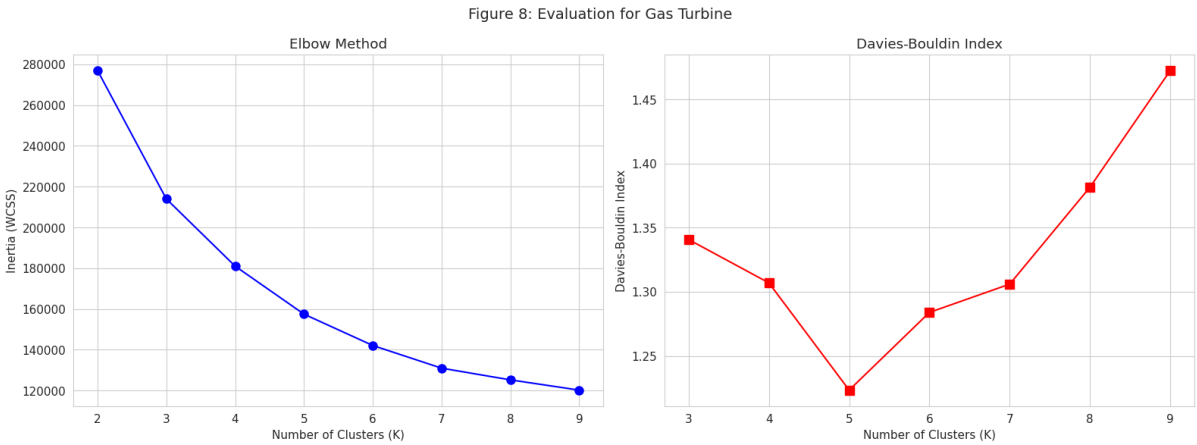

Figure 8: Elbow Method and Davies-Bouldin Index for choosing K

With this dataset, we start with the two figures which plot the evaluation with the Elbow Method and the Davies-Bouldin Index. From the Elbow Method, we can consider the value 4 for k, but this is not quite clear. Fortunately, with the DBI score, we realize that the value 4 is the local minimum value. From this, we consider using 4 for the value of K in the following process of the K-means clustering.

2.2. Explain the centroid initialization of the k-means used in the first question.

With the optimized value k=4 in the previous step, we can easily plot the 2D visualization of this dataset after performing PCA.

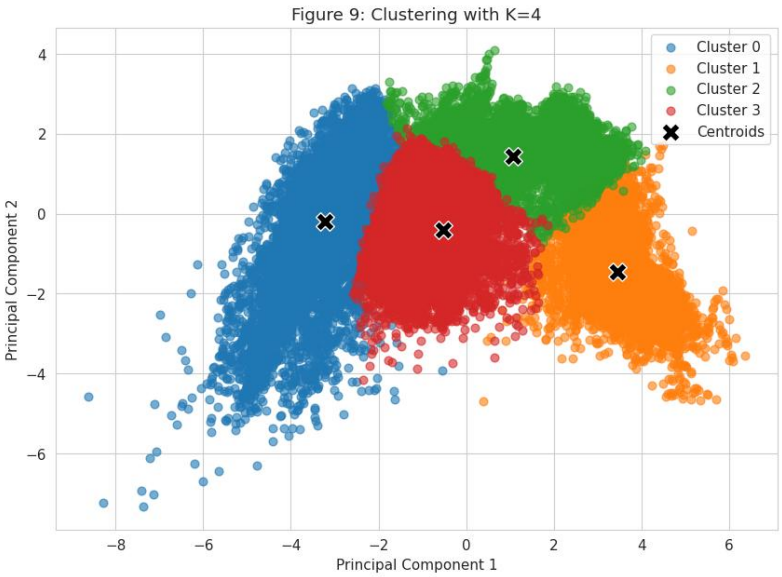

Figure 9: Data Visualization with k=4

2.3. Analyze and compare the results with different values of k.

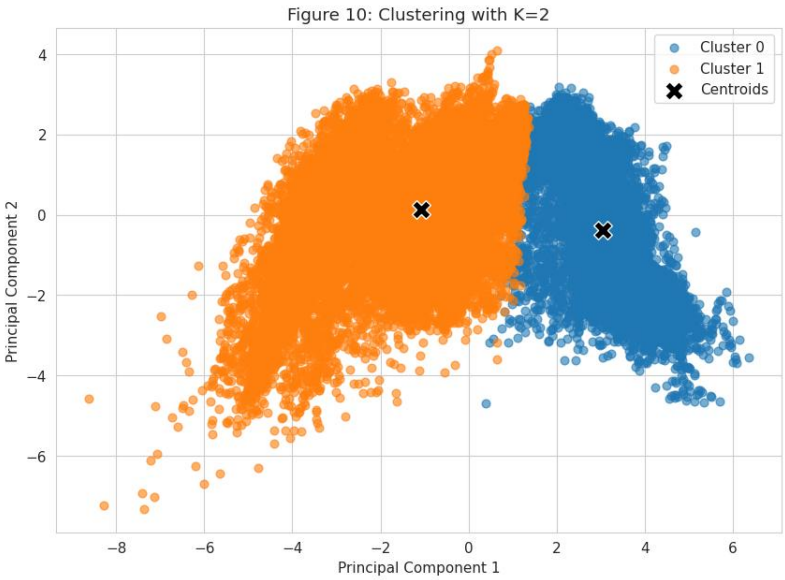

Figure 10: Data Visualization with k=2

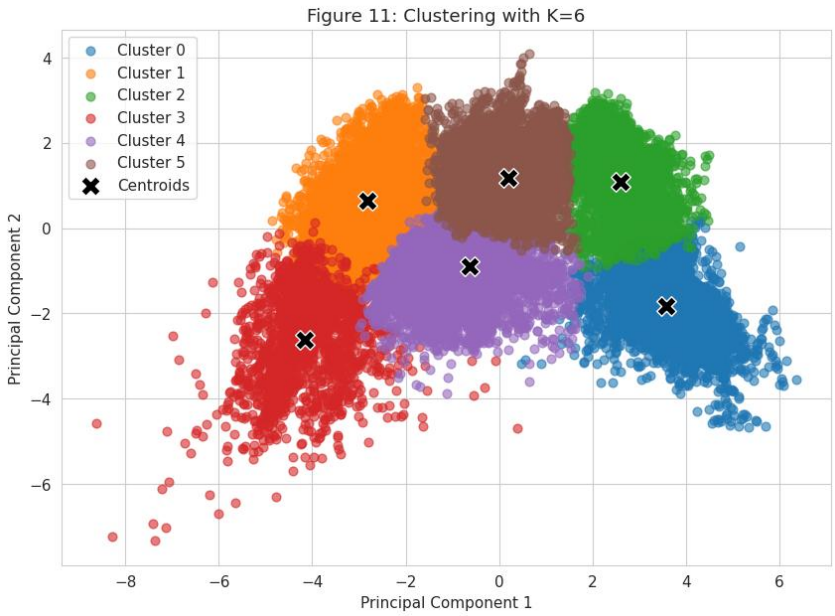

Figure 11: Data Visualization with k=6

As mentioned above, lower the DB score, the better clustering. k=4 achieve the lowest DBI score of 1.14 among three values of k we use to compare with each other. k=2 has close DBI values with 1.16 and k=6 has the highest DBI with the value of 1.31. With Inertia values, clearly k=6 has the lowest Inertia (3220.73) and it appears to be the best if we only consider Inertia. However, consider both metrics; k=6 has the best cluster but with DBI score, it shows that the separation between cluster is the worst among the values of K. Overall, k=4 is the best choice in this situation.

2.4. Calculate the clustering quality.

As mentioned in section 2.3, the DBI score for $k=4$ is 1.14 and it's the lowest among the other values of K. The table below shows more detail about DBI score Inertia.

Table 4: K-means clustering results (k=4) with vary max iter

max iter	Inertia	DBI Score
5	3938.52	1.14
10	3938.23	1.14
15	3938.23	1.14

As shown above, the clustering quality is stable very soon at iteration number 5 with Inertia of 3938 and DBI score 1.14 so trying to increase the maximum number of iteration beyond 5 isn't needed.

II. Subspace clustering

1. Dataset Overview

Dataset Detection of IoT botnet attacks - N BaIoT [3] was selected from UCI, which "addresses the lack of public botnet datasets, especially for the IoT. It suggests *real* traffic data, gathered from 9 commercial IoT devices authentically infected by Mirai and BASHLITE." The dataset has 89 separated files of the .csv format, with the total of instances is 7062606, 115 real number features and having no missing value.

For this dataset, we decided to use the K-Means method for the clustering task, which is the most suitable method for the large dataset.

2. Use PCA or SVD to visualize the data distribution in 2D/3D.

As mentioned before, this is a very large dataset with 7062606 instances, so applying the original PCA and K-Means will cost us an hour to complete. Luckily, Python provides better built-in functions for the process alternative, which are Incremental PCA (IPCA).

IPCA, as the name suggests, is an incremental version of PCA. It processes the data in batches (chunks) instead of loading the entire dataset as in PCA. The memory usage is proportional to the batch size and the number of components (n_components), not the total size of the dataset.

We start with applying the PCA process for this dataset. With some calculating and retrying in plotting the Scree Plot, we ended up with the Scree Plot with six first PCs and the last 115th PC for the readability and because of six first PCs, the total variance explained is about 80%.

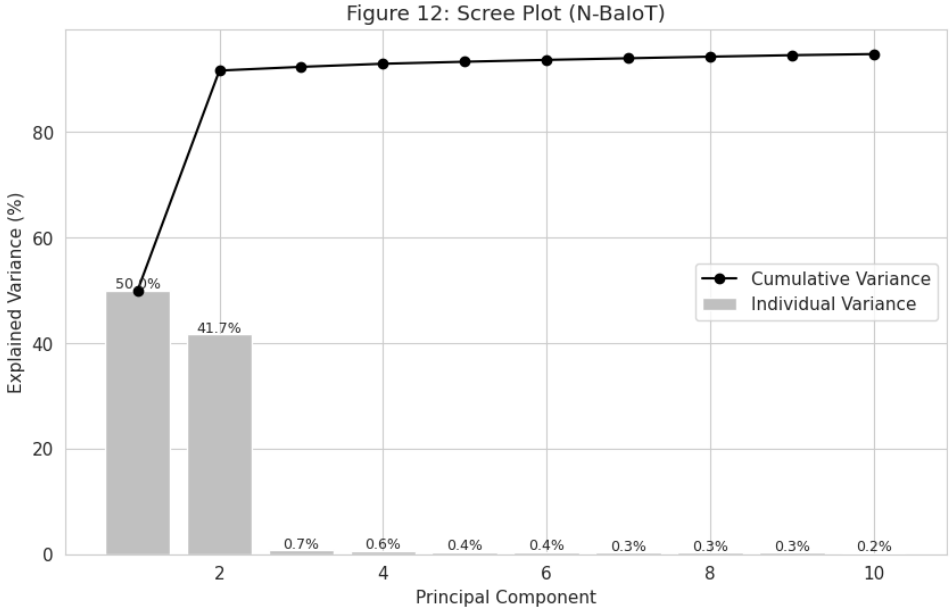

Figure 12: Scree Plot of N BaIoT Dataset

The Scree Plot illustrates that after PCA, the cumulative explained variance of the first 6 PCs is about 80%, and the first 2 PCs is approximately 50%. This is such a good values, the 2D visualization can capture the total variances very well.

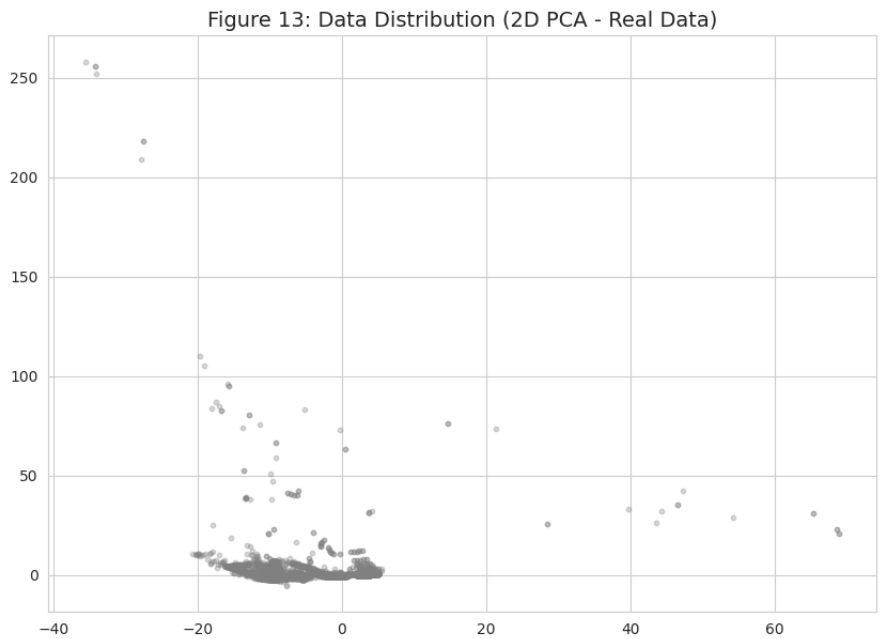

Figure 13: Data distribution visualization in 2D

3. Apply clustering method K-means for the selected datasets in 2D/3D after using PCA/SVD. Compare the performance before and after the dimensionality reduction.

Instead of using all the data at once like regular K-means, MiniBatchKMeans uses a random, small group of data points (a "mini-batch"). It only assigns the points in this mini batch to the closest cluster center, which makes it much faster.

Although we are not able to implement PCA and the K-Mean method for the evaluation, however, IPCA in combination with MiniBatchKmeans will bring back an approximate result, usually very close to PCA.

Starting with the figure plotting the evaluation for choosing K with the Elbow Method and the DBI. In this step, we use the entire dataset using MiniBatchKMeans, and the IPCA with number of $PC=2$ for taking the first two PCs for the most explained variance.

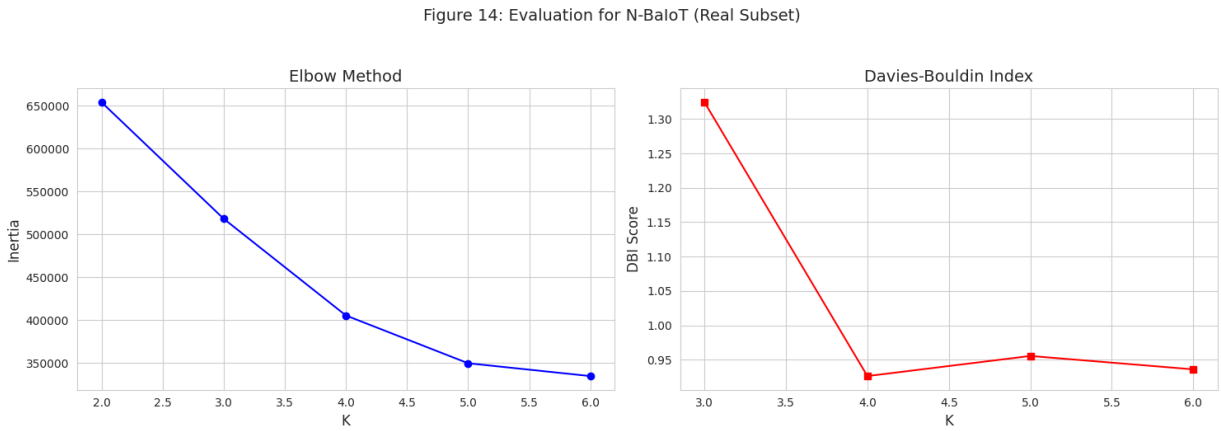

Figure 14: Choosing K with the entire dataset

With the figure of two evaluation methods which are the Elbow and DBI, we consider value 3 for k, which is not the best optimized Inertia but the local optimum value of DBI, So we will use k=3 in the next steps.

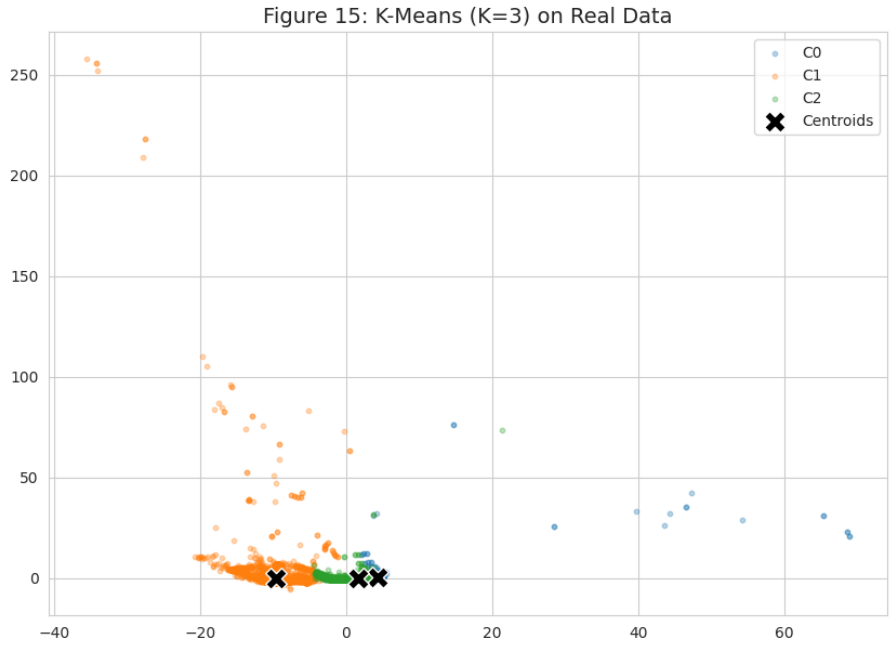

Figure 15: K-Means with the entire dataset visualization in 2D

Beside processing K-Means after PCA, we also do it without PCA with the very specific result as following:

Table 5: K-means results with/without PCA

Metric	Inertia	DBI	Execution Time (s)
With PCA	406,290,387	0.676	1.23
Without PCA	502,156,460	0.828	6.22

From the table with the very big and various values, we can also easily give the comment that, with PCA, the value of Inertia reduces significantly, which means that the compactness of each cluster is improved. With DBI values, less is more how well the clusters are separated. Moreover, the execution time is reduced 5 times with PCA, which is only measured in the calculation of DBI values step.

4. Select a random subspace of the dataset. Re-apply any clustering method on the subspace.

From the original dataset, we split it and get 10% for the sample. After getting the data frame, we also start plotting a figure to check and choose K again if there has some difference:

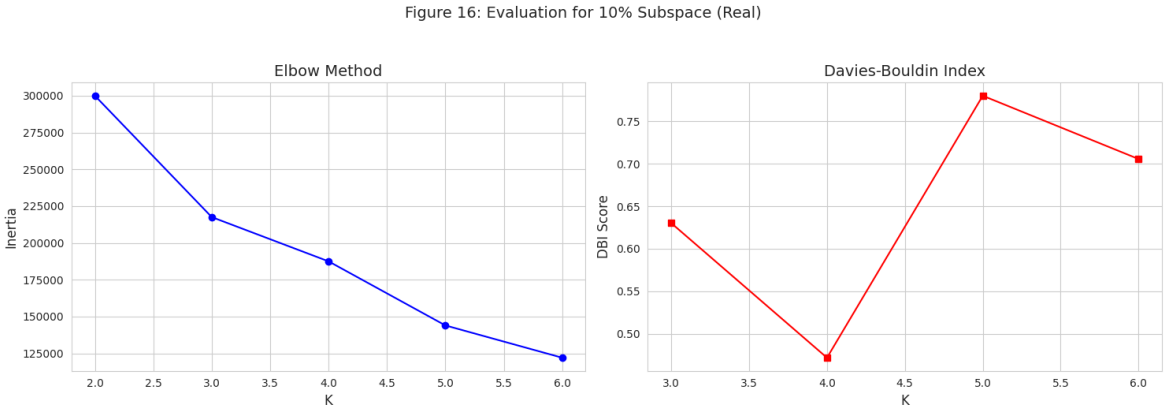

Figure 16: Choosing K with the 10% sample dataset

From the figure, the number of k=3 now is not the optimum value in the Elbow Method. We will try with the new k=4 for this mini sample dataset. With the random sample 10% selected in the dataset, we can save much more time in applying the clustering method. We plot the K-Means Clustering result with as following:

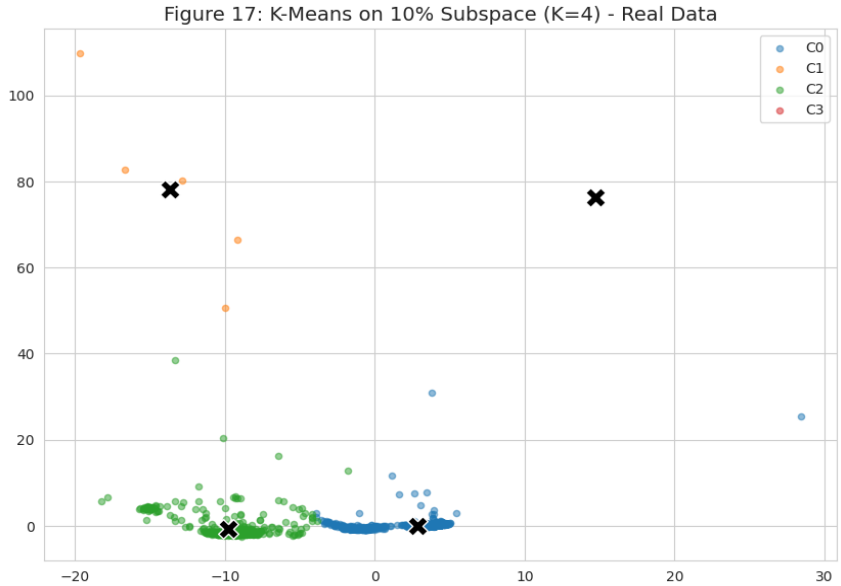

Figure 17: K-Means 2D visualization of the sample 10% dataset

The figure shows the clustering process is quite good, the ICs are significantly separated. However, there was too much outlier, so the Inertia value is very terrible. As the Inertia value measures how compact the clusters are, we can handle this situation by processing the outlier values.

Table 6: Evaluation comparison

Dataset	Inertia	DBI
Entire	7,062,038.83	0.53
Sample	140,059.24	0.40

The table compares the result of using K-Means clustering method with the original dataset, and the sample dataset, which is taken by randomly selecting 10% of the original dataset.

Discuss and compare the performance with the previous results.

With the original dataset, the complete execution of the clustering process cost us very much time, around 20-30 minutes. However, with the 10% sample dataset, full implementation will only be completed in about 5-7 minutes. With the sample dataset, the 2D visualization is quite similar with the original data, but the detail. The Inertia and DBI values are also better.

In conclusion, with this large dataset, we think that it is still good to choose the sample of data for clustering tasks because of its performance per time cost.

III.REFERENCES

[1] "Gas Turbine CO and NOx Emission Data Set," UCI Machine Learning Repository, University of California, Irvine. [Online]. Available: https://archive.ics.uci.edu/dataset/551/gas+turbine+co+and+nox+emission+data+set. [Accessed: Dec. 10, 2023].UCI Machine Learning Repository: Wine Recognition Data Set.

[2] "Land Mines-1," UCI Machine Learning Repository, University of California, Irvine. [Online]. Available: https://archive.ics.uci.edu/dataset/763/land+mines-1. [Accessed: Dec. 10, 2023].

[3] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, D. Breitenbacher, A. Shabtai, and Y. Elovici 'N-BaIoT: Network-based Detection of IoT Botnet Attacks Using Deep Autoencoders', IEEE Pervasive Computing. Special Issue - Securing the IoT (July/Sep 2018).

LIST OF ABBREVIATIONS

PC	Principal Component
IC	Initial Centroid
WCSS	Within-Cluster Sum of Squares
DBI	Davies-Bouldin Index